PriLearner: Teach Large Language Models to Use Private Library Effectively

Yitong Zhang* Chengze Li* Ruize Chen*

Jia Li† Yijie Ren Xiaoran Jia

Abstract

This is only the tempoary version of the paper, some cotents may be revised and some authors may be added. Large language models (LLMs) are increasingly used for code generation in industry, where a large fraction of development depends on internal *private* libraries. Because such libraries are typically absent from public pre-training corpora, even strong code LLMs often fail to invoke private APIs correctly. Existing solutions largely rely on retrieval-augmented prompting to expose API documentation at inference time, but our analysis shows that *seeing* the right specifications is frequently insufficient: models still omit required APIs or misuse them.

We present *PriLearner*, a scalable training framework that teaches LLMs to *use* private-library APIs effectively. *PriLearner* synthesizes supervision at scale via (i) compositional data synthesis that fuses atomic tasks into realistic requirements involving multiple APIs, and (ii) multidimensional quality verification that filters samples by syntax, execution with auto-generated tests, and an LLM-as-a-judge for overall reasonableness. We construct two new benchmarks based on recently released libraries, `NdonnxEval` and `NumbaEval`, and evaluate three popular code LLMs. Across settings, *PriLearner* substantially improves functional correctness for private-library-oriented code generation and is complementary to retrieval-based methods, while preserving general code generation performance on `HumanEval`.

CCS Concepts

• **Do Not Use This Code → Generate the Correct Terms for Your Paper**; *Generate the Correct Terms for Your Paper*; Generate the Correct Terms for Your Paper; Generate the Correct Terms for Your Paper.

Keywords

Do, Not, Use, This, Code, Put, the, Correct, Terms, for, Your, Paper

ACM Reference Format:

Yitong Zhang, Chengze Li, Ruize Chen, Jia Li, Yijie Ren, and Xiaoran Jia. 2018. *PriLearner*: Teach Large Language Models to Use Private Library Effectively. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 9 pages. https://doi.org/XXXXXXX.XXXXXXX

*Equal contribution.

†Corresponding author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference acronym 'XX, Woodstock, NY

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/2018/06

https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Private-Library-Oriented Code Generation refers to the automatic generation of code that leverages APIs provided by private libraries to meet specific coding requirements. This task plays a critical role in real-world software development. For instance, a significant portion of development in many companies is highly dependent on internal private libraries. Recently, Large Language Models (LLMs) have been widely adopted to support software automation. However, since private libraries are typically absent from public training corpora, LLMs often lack the necessary knowledge to use these APIs effectively. Consequently, their performance on private-library-oriented code generation remains limited, which restricts the practical impact of LLMs in real-world software development.

To mitigate this deficiency, a significant body of research has focused on injecting API knowledge directly into the model context. These approaches typically rely on Retrieval-Augmented Generation (RAG) techniques to recommend relevant API specifications from API documentation. By injecting this information into the context, they aim to temporarily equip the model with the required knowledge at inference time. While such approaches can improve LLM performance on private-library-oriented code generation to some extent, they primarily target the accuracy and completeness of the injected API Knowledge. As a result, a fundamental question remains underexplored: ***given the required API knowledge, can LLMs actually utilize them effectively?*** To answer this question, we conducted preliminary experiments in Section 3. We find that even when provided with detailed specifications of all required APIs, the model still fails to effectively utilize this ideal API knowledge to fulfill the coding requirements. These results suggest that the bottleneck is not merely injecting the right API knowledge, but also *using* the APIs effectively during generation.

In real-world software engineering practice, human programmers typically undergo systematic training when they join a company, learning how to *use* internal private libraries and *internalizing* the correct usage of APIs. Inspired by this, we believe that training models to effectively *use* private library APIs could fundamentally resolve the bottlenecks of private-library-oriented code generation. A central practical question in this direction is how to obtain training data at scale. Manually curating training data is labor-intensive and often infeasible. We therefore turn to LLM-driven data synthesis, where an LLM synthesizes their own training data that are subsequently used for Supervised Fine-Tuning (SFT). Unfortunately, because the LLM lacks prior knowledge of the private library, direct synthesis often yields poor results, primarily due to the following two challenges: ❶ ***Low Data Authenticity.*** In real development,

coding requirements typically necessitate the complex composition of multiple private APIs, whereas an LLM unfamiliar with the private library tends to produce overly simplified requirements that can be satisfied with only a small number of API calls. This leads to a significant mismatch with the complexity of practical development scenarios. ❷ ***Poor Data Quality.*** Limited knowledge of private library makes the LLM prone to synthesis data containing critical flaws, such as syntax errors or semantic hallucinations. Directly training on such noisy samples can be counterproductive, and may even substantially degrade the model's overall capabilities.

To address these challenges, we propose *PriLearner*, a scalable training framework that systematically teaches LLMs to *use* private-library APIs, rather than merely *seeing* API specifications in context. At its core, *PriLearner* consists of two key components: ❶ First, ***Compositional Data Synthesis*** improves data authenticity by fusing simple synthesized training samples into complex training samples, where the synthesized requirement demands coordinated use of multiple private APIs, better reflecting real-world development. ❷ Second, ***Multidimensional Quality Verification*** improves data quality via a multidimensional filtering pipeline: we (i) eliminate samples with syntactic errors, (ii) validate executability via auto-generated tests and execution, and (iii) further assess overall reasonableness using an LLM-as-a-Judge. By combining compositional data synthesis with multidimensional quality verification, *PriLearner* can synthesize high-authenticity and high-quality training data at scale, enabling LLMs to effectively master private-library usage without human intervention.

Evaluating private-library-oriented code generation requires libraries that are unseen to the model. However, many existing benchmarks are built on libraries that have been publicly released for a long time, making them unreliable for assessing the private-library-oriented generation capability of today's popular LLMs. To obtain a more faithful evaluation, we select two recently released libraries and carefully construct two new benchmarks, `NdonnxEval` and `NumbaEval`. We evaluate three widely used models along with several representative baselines on both benchmarks. Our results show that directly injecting the required API knowledge into the context does not reliably enable models to use the APIs effectively. In contrast, our approach enables models to internalize API knowledge and substantially improves functional correctness. Moreover, *PriLearner* is compatible with existing techniques and, when combined with them, achieves performance that significantly exceeds baselines. Additionally, these gains come with negligible impact on the models' general capabilities. In summary, our contributions are threefold:

- We show that injecting required API knowledge into the context is often insufficient for private-library-oriented code generation, and models must learn how to use them effectively.
- We propose *PriLearner*, a scalable training framework that helps LLMs internalize private-library knowledge and systematically improves their API usage.
- We carefully construct two novel benchmarks (*i.e.,* `NdonnxEval` and `NumbaEval`) to support rigorous evaluation. Extensive experiments on multiple models and baselines demonstrate the effectiveness of *PriLearner*.

Table 1: *pass@k* (%) on `NumbaEval` and `NdonnxEval`, where $k \in \{1, 3, 5\}$.

Method	NumbaEval			NdonnxEval		
	pass@1	*pass@3*	*pass@5*	*pass@1*	*pass@3*	*pass@5*
`DeepSeek-Coder-6.7B-Instruct`						
Vanilla	10.59	21.26	27.82	20.36	29.16	33.26
Oracle	17.97	34.23	42.45	38.05	50.82	55.86
Gain (↑)	**+7.38**	**+12.97**	**+14.63**	**+17.69**	**+21.66**	**+22.60**
`LLaMa3.1-8B-Instruct`						
Vanilla	8.13	16.97	21.73	15.86	23.50	26.55
Oracle	13.10	26.73	33.66	30.36	42.79	47.45
Gain (↑)	**+4.97**	**+9.76**	**+11.93**	**+14.50**	**+19.29**	**+20.90**

2 Background and Related Work

2.1 Large Language Models for Code Generation

In recent years, large language models have been widely adopted for code generation, enabling more efficient and automated software development. Popular model families, including Qwen, DeepSeek, LLaMa, and GPT, have demonstrated strong performance across a range of code generation benchmarks. However, once training is complete, a model's internal knowledge is fixed, which limits its ability to solve tasks that require knowledge not covered during training, such as private-library-oriented code generation.

2.2 Private-Library-Oriented Code Generation

Private-Library-Oriented Code Generation refers to generating code that leverages private-library APIs to satisfy specific coding requirements. Unlike conventional library-oriented code generation, models typically have little to no prior knowledge of the target library, where such private libraries are widely used within companies but are rarely included in public training corpora. To enable models to utilize private libraries effectively, a significant body of research has been proposed. For example, DocPrompting and APIFinder adopt straightforward Retrieval-Augmented Generation (RAG) to retrieve relevant APIs and include their information in the prompt. CAPIR and EpiGEN improve retrieval accuracy by decomposing coding requirements into some subtasks before retrieving API information. ExploraCoder further incorporates execution feedback to mitigate issues caused by ambiguity in API documentation. Overall, most existing approaches focus on injecting more complete and accurate API knowledge in the context, so that the model can ***see*** sufficient information at inference time, while paying less attention to teaching the model to ***use*** private-library APIs effectively.

3 Motivation

Most prior work on private-library-oriented code generation seeks to provide LLMs with more complete and accurate *required knowledge* at inference time, typically by retrieving API documentation and injecting it into the prompt. This line of work implicitly assumes that once the model is presented with required API knowledge, it can reliably apply it to satisfy the coding requirement. In this section, we challenge this assumption and show that *seeing* the right API knowledge does not necessarily translate into *using* it correctly.

To disentangle the effect of knowledge injection from retrieval quality, we design an oracle setting that eliminates retrieval errors.

Specifically, for each problem, we construct an *oracle* prompt that includes the full specifications of all APIs required to solve the task, including their signatures and derailed functional descriptions. This oracle knowledge represents an idealized upper bound for approaches that directly inject API knowledge into the model context. We compare it with a *vanilla* setting that uses the same base prompt but without any injected API documentation. Experiments are conducted on two benchmarks, `NumbaEval` and `NdonnxEval` (described in Section 6.2), using DeepSeek-Coder-6.7B-Instruct and LLaMa3.1-8B-Instruct.

Table 1 reports *pass@k* on `NumbaEval` and `NdonnxEval`. Although providing perfect API specifications consistently improves performance, the absolute gains remain far from sufficient for practical deployment. For instance, on `NumbaEval`, oracle prompting only increases *pass@1* from 10.59% to 17.97% for DeepSeek-Coder-6.7B-Instruct, and from 8.13% to 13.10% for LLaMa3.1-8B-Instruct, leaving most instances unsolved. On `NdonnxEval`, the oracle setting yields similarly limited improvements, with *pass@1* increasing by only 17.69% and 14.50% for the two models. Overall, these results suggest that directly injecting the required API knowledge into the context at inference time is insufficient for private-library-oriented code generation.

We further analyze the failed cases and find that errors are largely attributable to *ineffective utilization* of the provided oracle specifications, rather than missing required knowledge in the context. In many instances, the model either does not invoke a necessary API even when it is explicitly available, or invokes it in a semantically incorrect manner. We highlight two representative cases in Figure 1. ① In the first case, the task requires converting an array into an `ndonnx.Array` via `ndonnx.asarray` before applying `ndonnx.multiply`. Despite being provided with both API specifications, the model directly calls `ndonnx.multiply` on the raw input and omits the required conversion, which triggers a type error at runtime. ② In the second case, the task requires safely handling division-by-zero by using `ndonnx.where` to select between the computed quotient and a fallback value. Although the model attempts to use `ndonnx.where`, it misinterprets the API signature and calls `ndonnx.where` with only one argument, leading to an argument error and failing to implement the intended conditional replacement.

Takeaway: Even with oracle API knowledge, LLMs frequently fail to apply APIs actively and correctly. These findings suggest that the key bottleneck is enabling LLMs to *use* private-library APIs effectively, rather than merely *seeing* required API knowledge in context.

Case 1:

Input:

Goal: *Apply a scalar gain to every element of an array, returning a scaled output with the same shape.*

Required API Knowledge (Oracle) :

- ***ndonnx.asarray****(obj, *, dtype=None, device=None, copy=None) -> Array:*
  Converts an array-like object (e.g., NumPy array, scalar, nested sequence) into an ***ndonnx.Array****.*
- *ndonnx.multiply(x1, x2) -> Array:*
  Performs elementwise multiplication between two inputs and returns an ndonnx.Array. ***At least one input must be an ndonnx.Array****.*

Output:

```
def apply_gain(x: Array, gain: float)
    return ndonnx.multiply(x, gain)
```

Error:

At least one argument to 'multiply()' is of type 'ndonnx.Array'.

Case 2:

Input:

Goal: *Compute the elementwise ratio between two arrays, with an option to replace divisions by zero with a specified fallback value.*

Required API Knowledge (Oracle) :

- *ndonnx.where(****cond, a, b****) -> Array:*
  Elementwise selection: returns a where cond is true, otherwise b ...
- *...*

Output:

```
def safe_ratio(x: Array, y: Array, fallback=0.0):
    x = ndonnx.asarray(x)
    y = ndonnx.asarray(y)
    quotient = ndonnx.divide(x, y)
    zero_denominator = ndonnx.where(y == 0)
    quotient = ndonnx.where(zero_denominator, fallback, quotient)
    return quotient
```

Error:

'where()' missing 2 required positional arguments: 'a' and 'b'.

Figure 1: Two failed cases on `NdonnxEval` with Deepseek-Coder-6.7B-Instruct. Despite having access to oracle required API knowledge, the model fails to utilize it effectively.

4 Methodology

4.1 Overview

Our approach is motivated by a common practice in industry. When human programmers join a company, they rarely become productive by consulting API documentation only when needed. Instead, they undergo systematic training and repeated practice, gradually internalizing how to correctly and efficiently use private-library APIs. By analogy, we argue that LLMs should move beyond merely *seeing* more accurate API knowledge at inference time. They must also *internalize* this knowledge through training so that they can *use* private APIs reliably during generation.

To this end, we propose *PriLearner*, a framework that leverages an LLM to synthesize private-library training data at scale and subsequently uses this data to perform SFT on the model itself, as shown in Figure ??. Formally, let $\mathcal{L}$ denote a private library with an API set $\mathcal{A} = \{a_1, \ldots, a_{|\mathcal{A}|}\}$ and corresponding specifications $\mathcal{S} = \{s(a) \mid a \in \mathcal{A}\}$, where $s(a)$ includes the signature and functional description of API a. Our goal is to learn a model p_θ that, given a coding requirement r, generates a solution program y that is functionally correct and uses private APIs effectively. To train p_θ, *PriLearner* first constructs a synthetic dataset $\mathcal{D}_{\text{syn}} = \{(r_i, y_i)\}_{i=1}^{N}$, where each instance contains a requirement r_i and a reference solution y_i. We then fine-tune the model with the standard maximum-likelihood objective:

$$\min_{\theta} \mathcal{L}_{\text{SFT}}(\theta) = -\sum_{(r,y)\in\mathcal{D}_{\text{syn}}} \log p_\theta(y \mid r). \tag{1}$$

Although direct LLM-based synthesis of $\mathcal{D}_{\text{syn}}$ is appealing, it is unreliable in our setting because the model initially lacks private-library knowledge. As discussed in Section 1, naive synthesis therefore tends to produce data with low authenticity and poor quality. *PriLearner* addresses these challenges through two core components. First, ***Compositional Data Synthesis*** (Section 4.2) improves authenticity by constructing more realistic tasks that require coordinated use of multiple private APIs. Second, ***Multidimensional Quality Verification*** (Section 4.3) improves data quality via systematic filtering along syntax, executability, and overall reasonableness. Together, these components yield high-authenticity and high-quality training data, which we use to optimize Eq. 1. Finally, the resulting model can be deployed for private-library-oriented code generation (Section 4.4).

4.2 Compositional Data Synthesis

Challenge: Low Data Authenticity. In practical software development with private libraries, coding requirements often require nontrivial compositions of multiple private APIs to be solved. However, an off-the-shelf LLM has little to no prior exposure to the target private library, and therefore tends to synthesize trivial training instances whose solutions invoke only a small number of APIs. As a result, naively synthesized data provides limited supervision for learning how to coordinate private APIs in realistic development scenarios.

Method: Compositional Data Synthesis. To address this challenge, we propose Compositional Data Synthesis. The core idea is to first construct a pool of *atomic* training samples and then *fuse* multiple atomic samples into *complex* training sampless. These complex samples are designed to require the coordinated use of multiple private-library APIs, thereby mimicking the complexity of real-world tasks.

(1) Atomic Sample Synthesis. We first construct an atomic dataset $\mathcal{D}_{\text{atom}}$. Specifically, we randomly sample a subset of APIs from the library's API set $\mathcal{A}$ and retrieve their corresponding specifications $\mathcal{S}_{\text{seed}} \subset \mathcal{S}$ to serve as seeds. We then prompt the LLM to generate an atomic sample based on these seeds. Formally, this process can be represented as:

$$(r, y) = \text{Synthesis}(\mathcal{S}_{\text{seed}}, \mathcal{I}_{\text{atom}}), \tag{2}$$

where $\mathcal{I}_{\text{atom}}$ represents the instruction prompt for atomic synthesis (shown in Figure **??**), r is the generated coding requirement, and y is the corresponding reference solution. It is worth noting that since randomly sampled APIs may not always be relevant to each other, we do not strictly constrain the LLM to utilize every seed API in the generated solution. Repeating this process yields a collection of atomic samples $\mathcal{D}_{\text{atom}} = \{(r_i, y_i)\}$.

(2) Complex Sample Synthesis. Next, we construct complex samples by randomly selecting a small set of atomic samples $S_{\text{sub}} = \{(r_i, y_i)\}_{i=1}^{m}$ from $\mathcal{D}_{\text{atom}}$ and prompting the LLM to fuse them. This fusion process is defined as:

$$(r', y') = \text{Fuse}(S_{\text{sub}}, \mathcal{I}_{\text{comp}}), \tag{3}$$

where $\mathcal{I}_{\text{comp}}$ is the fusion instruction prompt (shown in Figure **??**). The function $\text{Fuse}(\cdot)$ instructs the LLM to (i) merge the multiple atomic requirements into a single, cohesive complex requirement r', and (ii) produce a reference solution y' that solves this new requirement. This results in the complex dataset $\mathcal{D}_{\text{comp}}$. Compared with $\mathcal{D}_{\text{atom}}$, $\mathcal{D}_{\text{comp}}$ contains training samples that involve the combinatorial usage of multiple APIs, which better reflects the authentic complexity of private-library-oriented development.

4.3 Multidimensional Quality Verification

Challenge: Poor Data Quality. In practice, although Compositional Data Synthesis can improve authenticity, the synthesized samples can be of low quality because the LLM lacks private-library knowledge. Without rigorous filtering, the resulting dataset is prone to critical issues, such as syntax errors, hallucinated API calls, incorrect API usage, and semantic mismatches between the requirement and the solution. Directly using such noisy samples for SFT can be counterproductive, as it may reinforce erroneous patterns and ultimately degrade the model's coding capability.

Method: Multidimensional Quality Verification. To address this challenge, we introduce ***Multidimensional Quality Verification***, an automatic pipeline that filters out low-quality samples immediately after synthesis. Given a synthesized pair (r, y), we retain it only if it passes three complementary checks that target different failure modes: syntactic correctness, execution validity, and overall reasonableness. Formally, we define three binary validators $V_{\text{syn}}, V_{\text{exe}}, V_{\text{sem}} \in \{0, 1\}$ and keep a sample only if

$$V(r, y) = V_{\text{syn}}(r, y) \cdot V_{\text{exe}}(r, y) \cdot V_{\text{sem}}(r, y) = 1. \tag{4}$$

(1) Syntactic Verification. We first perform lightweight static verification to ensure the basic well-formedness of the generated code. Specifically, we parse the reference solution y using the standard Abstract Syntax Tree (AST) parser of the target programming language. Any sample that fails to parse is discarded. This step removes obviously invalid code before invoking more expensive verification procedures.

(2) Execution Verification. Syntactic correctness does not imply correct private-API usage. To better ensure functional validity and to eliminate API hallucinations, we verify executability via test-based execution. For each sample (r, y), we prompt the LLM to generate a set of unit tests $T = \{t_j\}$, including representative inputs and corresponding assertions. We then execute y against T in a sandboxed environment with the private library installed. Samples are discarded if they raise runtime errors or fail any assertion. This step filters out a large fraction of incorrect API calls and many subtle misuse cases that are not detectable by syntax checks alone.

(3) Overall Reasonableness Verification. Even if a sample is executable, the synthesized requirement and solution may still be unreasonable. To further improve data quality, we employ an LLM-as-a-Judge to evaluate overall reasonableness. Concretely, we prompt the LLM to assess (i) whether the requirement r is realistic and well-defined, and (ii) whether the solution y truly satisfies r in intent. The prompting template is shown in Figure **??**. Any sample that fails to pass this evaluation is strictly discarded.

4.4 Training and Deployment

We construct the final training set $\mathcal{D}_{\text{syn}}$ by collecting all synthesized samples that pass Multidimensional Quality Verification (Eq. 4). We then fine-tune the model on $\mathcal{D}_{\text{syn}}$ using the SFT objective in Eq. 1.

After SFT on $\mathcal{D}_{\text{syn}}$, PriLearner can be directly applied to private-library-oriented code generation. Given a coding requirement r, the deployed model generates code as:

$$\hat{y} = \arg\max_{y} p_{\hat{\theta}}(y \mid r), \tag{5}$$

where the learned parameters $\hat{\theta}$ encode how to effectively use private-library APIs.

Moreover, we view PriLearner as orthogonal to prior work that injects required API knowledge into the model context. Existing approaches primarily improve how well the model can *see* the right API information at inference time, whereas PriLearner focuses on enabling the model to *internalize* and *use* such knowledge effectively. In practice, the two directions can be combined. Let $\text{Retrieve}(r)$ return relevant API knowledge. We can condition the fine-tuned model on the augmented prompt:

$$\hat{y} = \arg\max_{y} p_{\hat{\theta}}\big(y \mid [r; \text{Retrieve}(r)]\big), \tag{6}$$

where $[\,\cdot\,;\,\cdot\,]$ denotes concatenation.

5 Benchmark Construction

Evaluating private-library-oriented code generation requires benchmarks whose target libraries are unseen by the tested models. However, many existing benchmarks are built on long-released libraries that recent LLMs may have already fully learned. For example, the widely used `TorchDataEval` evaluates models on the TorchData library, which was released in December 2021, long before the knowledge cutoffs of most modern LLMs. This makes improvements on such benchmarks insufficient evidence that a method truly enhances private-library-oriented code generation.

To obtain a more rigorous evaluation, we construct two new benchmarks, `NdonnxEval` and `NumbaEval`. They are designed to reflect real-world private-library-oriented code generation settings, where the target libraries are largely absent from many modern model's training corpora. We detail the construction process below.

Library Selection. The core requirement is to select libraries that are unfamiliar to most tested models. We choose two libraries that were released in 2024 and underwent substantial development throughout 2025, namely `ndonnx` and `numba-cuda`. Both libraries are well maintained and provide detailed API documentation, which facilitates reliable benchmark construction and evaluation.

LLM-Driven Draft Generation. To reduce manual effort, we leverage GPT-5.2 to generate initial drafts, as it has already learned both libraries. For each library, we randomly sample a subset of API specifications and prompt the model to produce a draft instance consisting of a coding requirement and corresponding unit tests. We generate over 500 drafts for each library.

Manual Curation and Refinement. To guarantee benchmark quality, five student authors with extensive programming experience (3+ years on average) were tasked with filtering and refining the drafts. After over 100 collective person-hours of manual curation, we finalized 169 instances for `NdonnxEval` and 187 for `NumbaEval`. On average, each instance includes x test cases and requires the coordinated invocation of x distinct APIs, reflecting the complexity of authentic software development.

6 Experimental Setup

To assess PriLearner, we conduct extensive experiments designed to answer two Research Questions (RQs). In this section, we present the details of our experimental setup, including the evaluated benchmarks, evaluation metrics, comparison baselines, selected models, and other implementation specifics.

6.1 Research Questions

Our study aims to answer the following RQs:

RQ1: How does PriLearner perform in private-library-oriented code generation? This RQ verifies that PriLearner can effectively teach models to internalize and use private-library APIs. To answer this RQ, we apply several baselines and PriLearner to three LLMs and compare their performance across our constructed private-library benchmarks.

RQ2: Does PriLearner compromise the general code generation capabilities of the models? Updating model parameters carries the risk of degrading a model's foundational abilities. In this RQ, we evaluate the models trained with PriLearner on a widely used public code generation benchmark to assess whether their general code generation capabilities are preserved.

6.2 Benchmarks

We evaluate PriLearner on three benchmarks, including two newly constructed private-library benchmarks and one widely used public benchmark.

- `NumbaEval` and `NdonnxEval` (Section 5). We release two novel benchmarks for rigorous evaluation of private-library-oriented code generation. `NumbaEval` evaluates code generation with `numba-cuda`, and `NdonnxEval` evaluates code generation with `ndonnx`. They contain 169 and 187 instances, respectively.
- `HumanEval`. We include this existing benchmark to assess general code generation capability. It consists of 164 hand-written programming tasks and mainly involves built-in functions, without requiring any third-party libraries.

6.3 Metrics

We adopt *pass@k* and *exec@k* as our main evaluation metrics.

- ***pass@k.*** To measure functional correctness, we use *pass@k*. For each instance, we sample $n \geq k$ candidate solutions (we use $n = 10$ and $k \in \{1, 3, 5\}$), execute the provided test cases, and count the number of passing solutions c. We then compute *pass@k* using the standard unbiased estimator:

$$pass@k = \mathbb{E}_{\text{instances}}\left[1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}\right]. \tag{7}$$

- ***exec@k.*** As shown in Section 3, models often misuse private-library APIs and trigger runtime failures. We therefore report *exec@k* to measure basic executability. It is defined analogously to *pass@k*, except that a solution is counted as successful if it runs to completion on the test inputs without raising any exception.

6.4 Baselines

Our core contribution lies in enabling models to automatically learn how to *use* private libraries. Therefore, our fundamental baseline is the original model that has not undergone training with *PriLearner*, which we denote as ***Vanilla***.

Most existing methods for private-library-oriented code generation primarily focus on helping models *see* sufficient API knowledge at inference time by injecting retrieved documentation into the prompt. Although these methods are orthogonal to *PriLearner*, we include three representative approaches for comparison.

- ***Naive RAG.*** It adopts a straightforward retrieval-augmented generation pipeline to search for relevant APIs and include their specifications directly in the prompt.
- ***EpiGen.*** It utilizes an LLM to decompose complex coding requirements into subtasks and retrieves API knowledge for each subtask to improve the relevance of injected documentation.
- ***CAPIR.*** Building on task decomposition, *CAPIR* further reranks retrieved APIs using an LLM to improve the accuracy of injected API knowledge.

Furthermore, we include a baseline that represents the theoretical upper bound for approaches focused on helping models see the right API knowledge. In this setting, we explicitly provide the LLM with the specifications of all APIs required to solve the specific coding requirements. Consistent with Section 3, we term this upper-bound setting ***Oracle***.

6.5 Models

We evaluate *PriLearner* on three widely used LLMs that lack prior training exposure to the target libraries in `NumbaEval` and `NdonnxEval`: DeepSeek-Coder-6.7B-Instruct, Qwen2.5-Coder-7B-Instruct, and LLaMa3.1-8B-Instruct. For brevity, we refer to them as `DeepSeek-6.7B`, `Qwen-7B`, and `LLaMa-8B`, respectively.

6.6 Implementation Details

For *PriLearner*, to avoid confounding the results with potential distillation from a stronger model, we use each target model as the synthesizer to generate its own training data. For `numba-cuda`, we synthesize 6K training instances per model, and for `ndonnx`, we synthesize 20K training instances per model. The two synthesized datasets are used to train separate models for their corresponding benchmarks. We fine tune the models using LoRA with AdamW for one epoch and a batch size of 16.

For baselines that require task decomposition or reranking, we use the same LLM as the backbone for these steps. We adopt the widely used BGE-M3 as the embedding model for retrieval. For baseline-specific hyperparameters, we perform grid search on `NdonnxEval` and use the best-performing configuration.

To ensure fair comparison, all methods, including *PriLearner*, generate 10 samples per requirement with identical decoding settings, using temperature 0.5 and top-p 0.95. Additional details are provided in the *Supplementary Material*.

7 Experimental Results

7.1 RQ1: Effectiveness on Code Generation with Private Library

The primary objective of *PriLearner* is to improve models' ability to use private libraries. To assess this capability, we evaluate the effectiveness of *PriLearner* on private-library-oriented code generation benchmarks in this RQ.

Setting. We apply the baselines and *PriLearner* to the three models described in Section 6.5 and evaluate them on the two benchmarks introduced in Section 5. We report *pass@k* and *exec@k* as evaluation metrics, where $k \in \{1, 3, 5\}$. As discussed in Section 4.4, *PriLearner* is orthogonal to many existing approaches. Therefore, in addition to evaluating each baseline independently, we also combine *PriLearner* with these baselines following Eq. 6.

Results. The *pass@k* and *exec@k* of different approaches are reported in Table 2.

❶ *PriLearner substantially improves private-library-oriented code generation performance.* For example, on `NdonnxEval` with `Qwen-7B`, *PriLearner* increases *pass@1* from 23.37% to 47.99%, and improves *exec@1* from 28.52% to 55.86%. In contrast, EpiGen yields only marginal gains, improving *pass@1* and *exec@1* by 0.42% and 5.21%, respectively.

❷ *PriLearner consistently enhances model performance across different settings.* As shown in Table 2, *PriLearner* consistently improves both *pass@k* and *exec@k* under all evaluated settings. Conversely, existing retrieval-augmented approaches frequently degrade overall performance. For instance, on `NumbaEval` with `DeepSeek-6.7B`, *Naive RAG* decreases *pass@1*, *pass@3*, and *pass@5* from 10.59%, 21.26%, and 27.82% to 8.07%, 18.05%, and 24.94%, respectively. We attribute this degradation to the retrieval of irrelevant API knowledge, which may mislead generation.

❸ *PriLearner can readily surpass the theoretical upper bound of inference-time knowledge injection methods.* In most settings, *PriLearner* outperforms *Oracle*. For example, on `NdonnxEval` with `DeepSeek-6.7B`, *PriLearner* improves *pass@1* from 20.35% to 48.17%, whereas *Oracle* reaches only 38.05%. To further understand this gap, we conduct an additional analysis. Prior work suggests that model confidence can be measured by negative entropy, so we compute the average negative entropy of logits on generated API invocation statements. For `DeepSeek-6.7B` on `NdonnxEval`, the average negative entropy is -0.23 for *Vanilla* and only slightly improves to -0.21 under *Oracle*, indicating that even with all required API knowledge in context, the model gains limited confidence in API usage. In contrast, *PriLearner* increases it to -0.17, indicating that *PriLearner* more effectively improves the model's confidence to use private-library APIs to some extent.

❹ *Existing approaches can benefit from PriLearner and achieve superior results when combined with it.* Across all evaluated settings, combining existing approaches with *PriLearner* yields better results than using the corresponding baselines alone. For example, on `NumbaEval` with `LLaMa-8B`, *EpiGen* achieves *pass@5* of 25.32%, which increases to 42.28% when combined with *PriLearner*. Similarly, *Oracle* achieves *pass@5* of 33.66%, and further improves to 49.09% when combined with *PriLearner*.

Table 2: *pass@k* and *exec@k* (%) on `NdonnxEval` and `NumbaEval`.

Model	Method	NdonnxEval						NumbaEval					
		pass@1	*pass@3*	*pass@5*	*exec@1*	*exec@3*	*exec@5*	*pass@1*	*pass@3*	*pass@5*	*exec@1*	*exec@3*	*exec@5*
DeepSeek	*Vanilla*	20.35	29.16	33.26	25.74	37.09	42.20	10.59	21.26	27.82	22.25	42.34	52.23
	+PriLearner	48.17	60.36	64.92	61.72	74.19	78.58	24.12	38.47	45.04	52.73	74.26	82.21
	Naive RAG	14.97	25.16	29.87	25.15	38.75	43.85	8.07	18.05	24.94	17.38	38.04	50.71
	+PriLearner	46.80	60.60	65.60	62.49	76.03	80.58	19.30	32.66	39.01	43.48	66.74	75.76
	EpiGen	16.86	25.78	30.27	27.04	40.32	46.31	11.18	23.61	30.97	22.83	45.91	57.40
	+PriLearner	44.79	59.04	63.39	61.89	74.95	78.95	24.49	38.70	45.28	51.39	72.05	79.73
	CAPIR	18.76	29.22	34.15	28.88	41.27	45.72	10.32	21.52	27.82	20.00	40.64	51.32
	+PriLearner	47.10	60.01	65.04	62.37	75.90	80.57	24.28	38.30	44.81	53.85	74.81	81.54
	Oracle	38.05	50.82	55.86	51.01	67.34	72.96	17.97	34.23	42.45	30.32	55.73	66.82
	+PriLearner	55.44	67.56	71.79	71.01	82.86	86.94	28.66	43.24	48.88	55.19	77.48	84.32
Qwen	*Vanilla*	23.37	31.69	35.42	28.52	38.91	43.40	16.04	31.82	39.51	26.74	51.56	63.08
	+PriLearner	47.99	59.25	62.76	55.86	67.58	71.68	35.61	51.83	58.29	56.47	77.54	83.60
	Naive RAG	28.40	35.89	38.52	37.04	44.69	47.53	16.68	33.18	41.55	26.63	50.44	61.57
	+PriLearner	45.38	55.74	59.43	56.57	67.31	71.09	33.37	47.34	53.28	55.78	73.64	79.56
	EpiGen	28.82	35.69	38.00	33.73	41.48	44.46	16.47	32.75	40.70	27.43	52.33	63.15
	+PriLearner	47.63	56.49	59.58	57.93	68.09	71.50	35.08	49.74	56.69	56.84	75.43	81.06
	CAPIR	29.41	36.82	39.68	36.21	43.68	46.79	14.81	29.16	36.76	25.45	48.45	59.92
	+PriLearner	44.44	54.66	58.58	56.92	67.57	70.97	32.46	48.09	54.49	52.89	73.09	79.57
	Oracle	52.60	62.64	66.16	60.24	70.49	73.87	24.97	43.01	50.54	35.78	61.33	71.36
	+PriLearner	56.39	66.97	69.87	63.37	74.32	77.92	38.18	53.64	59.73	58.93	77.22	82.37
LLaMa	*Vanilla*	15.86	23.50	26.55	23.14	34.38	38.65	8.13	16.97	21.73	22.78	44.36	55.25
	+PriLearner	32.31	45.28	50.57	50.18	65.69	70.92	23.58	35.34	40.84	65.94	82.92	87.90
	Naive RAG	15.74	23.71	26.74	30.77	42.88	46.70	6.79	15.89	21.88	19.73	41.44	53.16
	+PriLearner	22.78	35.51	40.91	46.39	62.77	68.62	25.13	37.72	43.92	65.56	82.12	87.14
	EpiGen	11.72	19.86	22.93	23.55	38.04	43.94	9.57	19.55	25.32	24.97	48.97	60.37
	+PriLearner	27.51	41.07	46.99	45.80	61.75	68.08	23.96	36.33	42.28	64.49	82.02	87.04
	CAPIR	13.20	19.97	22.69	22.25	34.77	40.18	8.40	18.19	23.86	25.67	49.08	60.62
	+PriLearner	30.24	42.61	47.17	48.17	63.47	68.59	23.64	35.16	40.36	66.20	82.72	87.42
	Oracle	30.36	42.79	47.45	46.57	64.42	70.38	13.10	26.73	33.66	29.36	52.52	62.55
	+PriLearner	36.75	51.93	57.64	55.98	75.35	81.29	27.75	42.45	49.09	67.54	84.60	89.39

Answer to RQ1: *PriLearner* can effectively teach LLMs to use private-library APIs and yields substantial and consistent improvements in private-library-oriented code generation.

7.2 RQ2: Impact on General Code Generation

A practical concern of training-based approaches is that updating model parameters may hurt the model's general-purpose coding capability. In this RQ, we examine whether *PriLearner* introduces such side effects.

Setting. We evaluate the *PriLearner*-trained models on `HumanEval`, which consists of 164 Python programming tasks that do not require third-party libraries. To make a direct comparison with inference-time knowledge injection, we also run retrieval-based baselines on `HumanEval`, where the retrieved corpus is the API documentation of `numba-cuda`. For *PriLearner*, we use the models fine-tuned on synthesized data constructed for `numba-cuda` (i.e., the same training used for `NumbaEval`), and report *pass@1*, *pass@5*, *exec@1*, and *exec@5*.

Results. Table 3 reports the results on `HumanEval`. Overall, *PriLearner* preserves general code generation capability, while retrieval-based baselines often substantially degrade it.

❶ *PriLearner does not harm general code generation performance.* Across the three models, the change introduced by *PriLearner* is small. For example, on `HumanEval`, *PriLearner* changes *pass@1* by at most 2.07 points (for `DeepSeek-6.7B`), and slightly improves *exec@1* for all three models. These results suggest that fine-tuning with *PriLearner* on private-library data does not cause noticeable regression on general-purpose coding tasks.

❷ *Inference-time knowledge injection can be brittle on general tasks.* In contrast, retrieval-based baselines frequently introduce large performance drops. For instance, on `Qwen-7B`, *Naive RAG* reduces *pass@1* from 85.43% to 58.96% and reduces *exec@1* from 98.29% to 70.79%. We attribute this to irrelevant or distracting documentation being injected into the context for tasks that do not require the library at all.

❸ *Case study: irrelevant APIs may mislead generation.* Figure 2 shows a representative example. The *Naive RAG* prompt injects unrelated `numba-cuda` APIs, which leads the model to generate code that imports and calls CUDA utilities and fails all tests. Even without retrieval, the vanilla model produces an incomplete solution that misses necessary input-type checks. After training with *PriLearner*, the model generates a robust solution (e.g., explicitly checking that dictionary keys are strings) and passes all tests.

Table 3: *pass@k* and *exec@k* (%) on `HumanEval`. For retrieval-based baselines, the retrieved corpus is the API documentation of `numba-cuda`, while *PriLearner* is trained on synthesized data constructed for `numba-cuda`.

Method	*pass@1*	*pass@5*	*exec@1*	*exec@5*
		DeepSeek-6.7B		
Vanilla	71.89 (-0.00)	86.96 (-0.00)	93.60 (-0.00)	98.27 (-0.00)
Naive RAG	47.13 (-24.76)	70.88 (-16.08)	64.45 (-29.15)	88.65 (-9.62)
EpiGen	48.41 (-23.48)	72.88 (-14.08)	64.82 (-28.78)	89.54 (-8.73)
CAPIR	46.52 (-25.37)	71.29 (-15.67)	67.20 (-26.40)	86.90 (-11.37)
PriLearner	69.82 (-2.07)	85.92 (-1.04)	95.30 (+1.70)	98.37 (+0.10)
		Qwen-7B		
Vanilla	85.43 (-0.00)	92.85 (-0.00)	98.29 (-0.00)	99.95 (-0.00)
Naive RAG	58.96 (-26.47)	77.91 (-14.94)	70.79 (-27.50)	88.91 (-11.04)
EpiGen	58.23 (-27.20)	77.23 (-15.62)	69.51 (-28.78)	87.14 (-12.81)
CAPIR	53.17 (-32.26)	72.17 (-20.68)	65.91 (-32.38)	84.33 (-15.62)
PriLearner	85.67 (+0.24)	91.46 (-1.39)	98.90 (+0.61)	99.99 (+0.04)
		LLaMa-8B		
Vanilla	64.82 (-0.00)	78.17 (-0.00)	95.37 (-0.00)	99.29 (-0.00)
Naive RAG	46.59 (-18.23)	72.13 (-6.04)	75.67 (-19.70)	94.50 (-4.79)
EpiGen	46.40 (-18.42)	70.78 (-7.39)	78.84 (-16.53)	95.01 (-4.28)
CAPIR	39.88 (-24.94)	63.93 (-14.24)	66.34 (-29.03)	87.56 (-11.73)
PriLearner	65.12 (+0.30)	78.94 (+0.77)	96.46 (+1.09)	99.56 (+0.27)

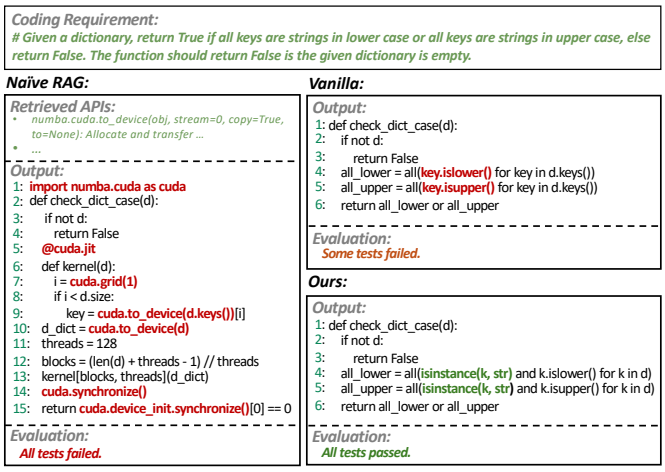

Figure 2: A representative `HumanEval` example where injected private-library documentation misleads retrieval-based baselines. Training with *PriLearner* preserves general coding behavior and produces a robust solution.

> **Answer to RQ2:** *PriLearner* preserves models' general code generation capability on `HumanEval`, while retrieval-based knowledge injection can substantially degrade performance when the injected documentation is irrelevant.

8 Discussion

8.1 Why internalizing private-library usage matters

Our motivation study (Section 3) suggests that the main bottleneck of private-library-oriented code generation is not only whether the model can *see* the required API specifications, but whether it can *apply* them correctly during generation. Inference-time knowledge injection methods are therefore inherently limited: even with oracle specifications in context, models may still omit required APIs or misuse them. By contrast, *PriLearner* trains the model to internalize private-library usage patterns through supervision, which can improve both the likelihood of invoking the right APIs and the confidence of generating correct invocations.

8.2 Practical considerations

Choice of synthesis model. In our setup (Section 6.6), we let each target model synthesize its own training data to avoid confounding the results with distillation from a stronger generator. In practice, one may use a more capable model as the synthesizer to increase the diversity and correctness of synthesized data, at the cost of additional compute and the possibility that improvements partially stem from cross-model distillation.

Cost and deployment. *PriLearner* is a one-time training effort per library, after which inference remains unchanged (Eq. 5) and can be combined with retrieval-based methods when documentation is available (Eq. 6). This makes the approach attractive for scenarios where a team repeatedly uses the same private libraries and can amortize synthesis and fine-tuning cost across many downstream tasks.

8.3 Limitations and future work

First, our evaluation focuses on Python libraries and two representative ecosystems; the effectiveness of *PriLearner* for other programming languages, library styles, or API surfaces (e.g., complex stateful clients) needs further study. Second, Multidimensional Quality Verification relies on auto-generated tests and an LLM-as-a-judge; both can be imperfect, and some incorrect samples may slip through or correct samples may be filtered out. Third, we currently train separate models per library. An interesting direction is multi-library training that shares representations across libraries, while avoiding interference between unrelated APIs.

8.4 Threats to validity

Construct validity. We use unit tests to approximate functional correctness, but tests may not fully cover edge cases. **Internal validity.** Although we strive to keep decoding settings and prompts consistent across methods, retrieval-based baselines are sensitive to retrieval quality and prompt formatting. **External validity.** Results are reported on three model families and two target libraries; generalization to other models and real-world private libraries may vary depending on documentation quality and API complexity.

9 Conclusion

Private-library-oriented code generation is a common and important setting in real-world software development, yet it remains challenging for modern LLMs because private APIs are rarely present in public training corpora. Our study shows that simply injecting API documentation at inference time is often insufficient: models may still fail to correctly use the provided APIs. To address this, we propose *PriLearner*, a scalable training framework that synthesizes high-authenticity, high-quality supervision through compositional

data synthesis and multidimensional quality verification, and then fine-tunes the model to internalize private-library usage.

We construct two new benchmarks, `NdonnxEval` and `NumbaEval`, and demonstrate that *PriLearner* substantially improves functional correctness on private-library-oriented code generation across multiple models, while preserving general code generation performance on `HumanEval`. We hope *PriLearner* and the released benchmarks will facilitate future research on training LLMs to reliably use private libraries in practical development.

References